CHATCO BACKEND SPRINT ROADMAP

Backend-Only Sprint Plan • June 16 – August 15, 2025

Laravel + Supabase | Sanctum Auth | GCash Direct QR | GPS + Hail System

Executive Summary

This roadmap covers 9 backend-only sprints from June 16 to August 15, 2025, targeting production readiness for the Chatco jeepney tracking system. The frontend (Next.js) is already implemented; this plan exclusively addresses the Laravel backend that the frontend expects to connect to.

Key architectural decisions confirmed throughout all sprints:

- Wallet system is fully and permanently removed — no internal balance, no top-up, no ledger
- GCash QR is direct external payment via PayMongo only — no internal money movement
- All jeepney units are visible to commuters at all times — no distance filter on visibility
- 1KM hail restriction is enforced server-side only — frontend is display-only
- Backend is the single source of truth — frontend never bypasses backend validation

Backend Architecture at a Glance

Layer	Implementation
API Framework	Laravel 11 (PHP 8.2+)
Database	Supabase (PostgreSQL) via Eloquent ORM
Authentication	Laravel Sanctum — SPA cookie-based + token fallback
Roles	ADMIN | CONDUCTOR | COMMUTER — enforced by middleware
GPS	POST /api/conductor/location → vehicle_locations table (upsert)
Hail Radius	Haversine server-side in HailService — 1000m hard limit

Payment	GCash via PayMongo API — no internal wallet or balance
Real-time	5s polling (MVP); Laravel Echo/Pusher optional upgrade

Weekly Sprint Breakdown

WEEK 1 June 16 – June 22, 2025

Sprint Goal: Laravel project bootstrap — auth skeleton, DB schema, Sanctum, role middleware

API & Route Work

- Install Laravel 11 + configure .env (DB, CORS, Sanctum, mail)
- Register all route groups: api/auth, api/commuter, api/conductor, api/admin, api/payments, api/qr
- Return 501 Not Implemented stubs for all non-auth endpoints (prevents silent failures)
- Standardize all API responses: { success, data, message, errors, meta } format

Auth & Security

- Install & configure Laravel Sanctum (cookie-based, SPA)
- **Implement POST /api/auth/login**
  - Validate email + password
  - Return Sanctum token or set httpOnly session cookie
  - Embed role claim in response: { id, email, role }
- Implement POST /api/auth/logout (revoke token)
- Implement GET /api/user (return authenticated user + role)
- Create role middleware: EnsureRole::class (admin, conductor, commuter)
- Protect all non-auth route groups with auth:sanctum + role middleware

Core System Logic

- No business logic this week — focus on foundation only
- Confirm roles enum in DB: ADMIN, CONDUCTOR, COMMUTER

Database / Laravel Structure

- Create migrations: users, conductors, commuters, vehicles, shifts, transactions, hails, remittances, announcements, lost_items, feedback
- Seed: 1 admin, 2 conductors, 3 commuters, 3 vehicles
- Create base models with $fillable, $hidden, $casts
- No wallet table — confirm it does not exist in any migration

Real-time / GPS

- No GPS work this week

- Create placeholder route: POST /api/conductor/location (returns 501)

Code Quality

- Folder structure: app/Http/Controllers/{Auth,Commuter,Conductor,Admin,Payment}, app/Services/, app/Http/Middleware/
- No business logic inside controllers — controllers call services only

Validation Checklist

- POST /api/auth/login returns token + role
- GET /api/user returns user behind auth:sanctum
- Non-auth routes return 401 when unauthenticated
- Role middleware blocks wrong-role access (test: commuter hitting /api/admin)
- No wallet table in DB schema
- All routes registered and returning 501 stubs (not 404)

WEEK 2 June 23 – June 29, 2025

Sprint Goal: Conductor shift + GPS location system — production-grade shift lifecycle

API & Route Work

- POST /api/conductor/shifts/start — create shift, link to conductor + vehicle
- POST /api/conductor/shifts/{id}/end — close shift, compute duration
- GET /api/conductor/shifts/active — return active shift for authenticated conductor
- GET /api/conductor/shift-logs — paginated shift history
- POST /api/conductor/location — accept { lat, lng, speed, heading } GPS payload
- GET /api/vehicles/locations — return all active vehicle locations (commuter-facing)

Auth & Security

- All conductor routes behind auth:sanctum + role:conductor middleware
- Conductor can only access their own shifts (scope by conductor_id = auth()->id())
- Reject duplicate active shifts: one conductor = one active shift max

Core System Logic

- Shift states: pending → active → ended (enforce server-side transitions)
- GPS update: store latest lat/lng per vehicle in vehicle_locations table (upsert by vehicle_id)
- Vehicles table: track active_shift_id FK for real-time status
- Capacity status: Available / Standing / Full — conductor updates via API, never inferred by frontend

Database / Laravel Structure

- Migration: vehicle_locations (vehicle_id, conductor_id, lat, lng, speed, heading, capacity_status, updated_at)

- Migration: shifts (id, conductor_id, vehicle_id, started_at, ended_at, status)
- ShiftService: startShift(), endShift(), getActiveShift()
- LocationService: updateLocation(), getAllActiveLocations()

Real-time / GPS

- POST /api/conductor/location: validate lat (-90 to 90), lng (-180 to 180), required fields
- Upsert vehicle_locations on every POST — always latest position only
- GET /api/vehicles/locations: return all vehicles with lat, lng, capacity_status, plate_number
- Frontend polls GET /api/vehicles/locations every 5s (no WebSocket needed this sprint)

Code Quality

- ShiftController → delegates entirely to ShiftService
- LocationController → delegates entirely to LocationService
- No raw DB queries in controllers

Validation Checklist

- Conductor can start exactly one active shift
- Second start attempt returns 409 Conflict
- GPS POST updates vehicle_locations (verify in DB)
- GET /api/vehicles/locations returns all vehicles regardless of distance
- Commuter role cannot POST /api/conductor/location (403 expected)
- Shift end sets ended_at timestamp

WEEK 3 June 30 – July 6, 2025

Sprint Goal: Hail system with server-enforced 1KM radius — backend is the sole gatekeeper

API & Route Work

- POST /api/commuter/hail — create hail request
- DELETE /api/commuter/hail/{id} — cancel hail
- GET /api/conductor/hails — conductor sees pending hails for their vehicle
- POST /api/conductor/hails/{id}/accept — conductor accepts hail
- POST /api/conductor/hails/{id}/reject — conductor rejects hail

Auth & Security

- Hail POST requires auth:sanctum + role:commuter
- Conductor hail actions require auth:sanctum + role:conductor
- Commuter can only cancel their own hail

Core System Logic

- **1KM Radius Enforcement (SERVER-SIDE ONLY)**
  - On POST /api/commuter/hail: receive { vehicle_id, commuter_lat, commuter_lng }
  - Fetch latest vehicle lat/lng from vehicle_locations
  - Compute Haversine distance server-side
  - If distance > 1000m → return 422 { error: 'outside_radius', distance_m: N }
  - If distance <= 1000m → create hail record
  - Frontend distance check is DISPLAY-ONLY and must never be the sole validator
- Hail states: pending → accepted | rejected | cancelled | expired
- Hail expiry: auto-expire pending hails after 3 minutes (queue job or GPS poll)
- Vehicle visibility: GET /api/vehicles/locations returns ALL vehicles — no radius filter here

Database / Laravel Structure

- Migration: hails (id, commuter_id, vehicle_id, conductor_id, commuter_lat, commuter_lng, distance_m, status, created_at, expires_at)
- HailService: createHail(), cancelHail(), acceptHail(), rejectHail(), expireStaleHails()
- HailRequest FormRequest: validate lat/lng ranges, vehicle_id exists and is active

Real-time / GPS

- Hail creation triggers distance validation against live vehicle_locations
- Conductor sees hails via GET /api/conductor/hails — scoped to their vehicle_id
- Consider: scheduled job every 60s to expire stale hails (Laravel scheduler)

Code Quality

- HailController is thin — all logic in HailService
- Haversine function in app/Helpers/GeoHelper.php (reusable, testable)
- Remove any 1KM logic from frontend API route stubs — it must only live in backend

Validation Checklist

- Hail at 0.5km succeeds (201 Created)
- Hail at 1.5km fails (422, error: outside_radius)
- Admin/conductor cannot POST /api/commuter/hail (403)
- GET /api/vehicles/locations still returns ALL vehicles even if commuter is far
- Conductor can accept/reject only hails for their vehicle
- Cancelled hail status = cancelled in DB

WEEK 4 July 7 – July 13, 2025

Sprint Goal: Transaction system + GCash QR payment flow (no wallet, direct payment only)

API & Route Work

- POST /api/conductor/transactions — record a fare transaction (cash or GCash)

- GET /api/conductor/transactions?shift_id={id} — list transactions for a shift
- POST /api/gcash/charge — initiate GCash direct payment (proxies to PayMongo)
- GET /api/payments/verify?id={id} — verify payment status
- POST /api/payments/webhook — PayMongo webhook for async payment confirmation
- GET /api/commuter/payments — commuter payment history

Auth & Security

- POST /api/gcash/charge requires auth:sanctum (conductor role)
- Webhook endpoint: verify PayMongo signature header (prevent spoofed callbacks)
- Commuter can only see their own payment history
- No admin can create transactions directly

Core System Logic

- **GCash Payment Flow (Direct, No Wallet)**
  - Conductor selects GCash for a commuter fare
  - Backend calls PayMongo: POST /payment_intents with amount in centavos
  - Return QR source URL to frontend for display
  - Commuter scans QR in GCash app
  - PayMongo calls /api/payments/webhook on completion
  - Backend updates transaction status to paid
  - NO internal balance, NO wallet credit, NO stored funds — payment only
- Cash transactions: record immediately, status = paid, no external API
- Transaction record must link: shift_id, conductor_id, commuter_id (if known), fare_amount, payment_method, status

Database / Laravel Structure

- Migration: transactions (id, shift_id, conductor_id, commuter_id?, amount, payment_method, status, paymongo_id?, paid_at, created_at)
- Confirm: no wallet_balance column on any table, no wallet_transactions table
- TransactionService: createTransaction(), verifyGCashPayment(), handleWebhook()
- PaymentService: createPaymentIntent(), verifyIntent() — wraps PayMongo API

Real-time / GPS

- Webhook: when payment confirmed → update transaction.status = paid + transaction.paid_at
- Frontend polls GET /api/payments/verify every 3s after QR display (fallback if webhook delayed)

Code Quality

- PayMongo API key stored in .env only — never in code
- PaymentService handles all external calls — TransactionController never calls PayMongo directly
- Webhook handler validates signature before processing any data

Validation Checklist

- Cash transaction records correctly in DB

- GCash flow: POST /api/gcash/charge returns QR data without creating wallet
- Webhook updates transaction status to paid
- Commuter payment history shows transactions
- No wallet table or wallet balance column in DB (confirm via schema inspection)
- PayMongo secret key is NOT hardcoded (check .env)

WEEK 5 July 14 – July 20, 2025

Sprint Goal: Commuter profile, admin CRUD, remittance, and role-scoped data access

API & Route Work

- GET/PUT /api/commuter/profile
- POST /api/commuter/change-password
- GET /api/conductor/remittances + POST /api/conductor/remittances
- GET /api/admin/users — paginated, filterable by role
- GET/PUT/DELETE /api/admin/users/{id}
- GET /api/admin/vehicles + POST + PUT/DELETE
- GET /api/admin/monitoring — live fleet data
- GET /api/admin/analytics — aggregated metrics

Auth & Security

- Commuter profile: user can only modify their own data
- Admin endpoints: all behind role:admin middleware
- Admin cannot impersonate users via API (no user-switching endpoint)
- Password change: validate current_password before allowing update

Core System Logic

- Remittance: conductor submits daily collection amount at end of shift
- Remittance links to shift_id — one remittance per shift
- Admin monitoring: aggregate from vehicle_locations + active shifts in real-time
- Analytics: aggregate transactions by date, route, payment_method (no wallet metrics)

Database / Laravel Structure

- Migration: remittances (id, conductor_id, shift_id, total_collected, remitted_amount, created_at)
- Admin views use DB aggregates — no fake/static data
- CommuterService, ConductorService, AdminService separation
- All services return typed data — controllers just format responses

Real-time / GPS

- GET /api/admin/monitoring: real-time vehicle positions from vehicle_locations
- Admin monitoring refreshes every 5s (same polling mechanism as commuter map)

Code Quality

- Remove any static/hardcoded admin data from backend stubs
- Ensure commuter endpoints never leak conductor/admin data
- Clean up any duplicated profile logic between AuthController and CommuterController

Validation Checklist

- Commuter cannot access /api/admin endpoints (403)
- Admin can list, update, delete users
- Remittance links to correct shift_id
- Profile update persists to DB
- Analytics endpoint returns real aggregated data, not mocked values

---

WEEK 6 July 21 – July 27, 2025

Sprint Goal: QR signing, feedback, lost & found, announcements, SOS alert

API & Route Work

- POST /api/qr/sign — generate signed QR payload for a transaction
- POST /api/qr/verify — verify QR signature on scan
- POST /api/commuter/feedback — submit ride feedback
- GET /api/lost-found + POST /api/lost-found
- PUT /api/lost-found/{id}/claim
- GET /api/announcements + POST (admin only)
- POST /api/commuter/sos — broadcast SOS alert

Auth & Security

- QR signing: HMAC-SHA256 with server secret key (never expose secret client-side)
- QR verify: reject tampered or expired QR payloads
- SOS: rate-limit to prevent abuse (max 1 per 5 minutes per commuter)
- Feedback: commuter only, linked to commuter_id

Core System Logic

- **QR Signing Flow**
  - Sign payload: { transaction_id, amount, commuter_id, exp } with HMAC
  - Verify: recompute HMAC, reject if mismatch or expired
  - QR is tied to a transaction — NOT a wallet funding source
- SOS: store alert with commuter_id, lat, lng, timestamp — admin notified via broadcast or push
- Announcements: admin creates, all commuters can read, mark-as-read per user
- Lost & Found: commuter claims item → status = claimed, admin confirms

Database / Laravel Structure

- Migration: feedback (id, commuter_id, conductor_id?, shift_id?, rating, comment, created_at)
- Migration: lost_items (id, description, reported_by, status, claimed_by?, created_at)
- Migration: announcements (id, title, body, created_by, created_at)
- Migration: announcement_reads (user_id, announcement_id, read_at)
- Migration: sos_alerts (id, commuter_id, lat, lng, status, created_at)

Real-time / GPS

- SOS alert may broadcast via Laravel Echo / Pusher to admin dashboard
- Announcements: commuters pull on login or dashboard load

Code Quality

- QR secret in .env only
- QrService: sign(), verify() — isolated, testable
- SOS rate limiter via Laravel's built-in throttle middleware

Validation Checklist

- Signed QR cannot be tampered (modify payload → verify rejects it)
- Expired QR rejected after TTL
- SOS stores alert in DB with correct lat/lng
- Feedback links to correct commuter_id
- Announcement mark-as-read scoped per user
- Lost item claim changes status to claimed

WEEK 7 July 28 – August 3, 2025

Sprint Goal: API hardening, error standardization, dead code purge, wallet logic elimination

API & Route Work

- Audit all routes: remove any 501 stubs that were never implemented
- Standardize all error responses: { success: false, message, errors: {} }
- Standardize all success responses: { success: true, data, meta }
- Add 404 catch-all for undefined API routes (return JSON, not HTML)
- Review and fix any inconsistent HTTP status codes (201 vs 200 for creates, etc.)

Auth & Security

- Audit all endpoints: confirm auth:sanctum + role middleware on every protected route
- Test CSRF protection on POST routes (Sanctum SPA config)
- Ensure no endpoint leaks data from other roles (test cross-role access systematically)
- Rotate any hardcoded secrets found in code to .env

Core System Logic

- **Wallet Logic Elimination Audit**
  - Search entire codebase for: wallet, balance, top_up, wallet_ledger, stored_funds
  - Remove any wallet-related DB columns, tables, routes, or service methods
  - Confirm GCash flow has NO internal balance side effect
  - Any voucher/reward system must NOT store monetary balance
- Validate that hail radius check is ONLY in HailService — remove from any route/controller
- Confirm vehicle visibility endpoint has NO distance filter

Database / Laravel Structure

- Run schema audit: verify no wallet_* tables exist
- Add missing indexes: vehicle_locations.vehicle_id, transactions.shift_id, hails.commuter_id
- Add DB-level constraints: shift.status ENUM, hail.status ENUM
- Review all FK relationships and add ON DELETE behavior

Real-time / GPS

- GPS update endpoint: add server-side staleness check (reject GPS older than 30s via timestamp)
- Vehicles with no location update in 10 min: auto-mark as inactive in admin monitoring

Code Quality

- Remove all TODO/FIXME comments that reference wallet logic
- Remove unused imports across all service files
- Ensure every controller method is ≤30 lines — extract to service if not
- Verify no raw SQL queries — all queries through Eloquent or Query Builder

Validation Checklist

- grep -r 'wallet' app/ returns zero results
- All API errors return JSON (not HTML exception pages)
- Cross-role access test matrix: all 403s where expected
- DB schema: no wallet-related tables or columns
- GPS stale data rejected correctly
- Postman collection passes all endpoint tests

WEEK 8 August 4 – August 10, 2025

Sprint Goal: Performance, query optimization, caching, and staging environment readiness

API & Route Work

- Add pagination to: /api/admin/users, /api/conductor/transactions, /api/commuter/payments
- Add filtering to admin endpoints: ?role=, ?status=, ?date_from=, ?date_to=

- Add rate limiting: 60 req/min for general API, 5 req/min for auth endpoints
- Review and document all API contracts (route, method, request, response)

Auth & Security

- Implement token expiry: Sanctum tokens expire after 24h (configure in sanctum.php)
- Add brute-force protection on login: lockout after 5 failed attempts
- Ensure all file uploads (ID photos) are validated for type + size before storage
- Review CORS allowed_origins: must not be '*' in staging

Core System Logic

- Hail expiry: ensure scheduler is running and expiring stale hails correctly
- Transaction idempotency: prevent duplicate transaction on double-POST (unique key or idempotency header)
- Webhook retry handling: PayMongo retries failed webhooks — ensure idempotent processing

Database / Laravel Structure

- Add composite indexes for common query patterns
- Review N+1 query risks: use eager loading (with()) in listing endpoints
- Set up DB connection pooling for staging
- Create staging .env.staging — separate DB, PayMongo sandbox keys

Real-time / GPS

- GPS polling: confirm 5s interval is sustainable at scale (estimate: 10 conductors = 2 req/s)
- Cache GET /api/vehicles/locations with 4s TTL (Redis or in-memory) to reduce DB hits
- Consider Laravel Echo + Pusher for real-time push if polling proves insufficient

Code Quality

- Run php artisan route:list — confirm all routes are intentional
- Run php artisan optimize — cache config, routes, views for staging
- Ensure all validation rules use FormRequest classes — no inline $request->validate()

Validation Checklist

- Pagination works on all list endpoints
- Rate limiting returns 429 after threshold
- Login lockout after 5 failures
- N+1 queries eliminated (test with Laravel Debugbar or telescope)
- Staging environment boots and connects to separate DB
- PayMongo sandbox webhook verified in staging

WEEK 9 August 11 – August 15, 2025

Sprint Goal: Production hardening, final security audit, deployment readiness sign-off

API & Route Work

- Final API contract freeze — no new endpoints after August 12
- Ensure all endpoints have Postman tests documented
- Confirm frontend API_BASE_URL environment variable is wired to Laravel backend

Auth & Security

- Run OWASP checklist: SQL injection (Eloquent protects — confirm no raw queries), XSS, CSRF
- Confirm Sanctum CSRF cookie is set correctly for production domain
- Review sensitive data in logs: ensure passwords, tokens, PayMongo keys never logged
- Set APP_DEBUG=false in production .env
- Set APP_ENV=production

Core System Logic

- Final end-to-end flow tests: commuter hail → conductor accept → GCash payment → transaction confirmed
- GPS update → vehicle visible on commuter map → hail blocked at 1.5km → hail allowed at 0.5km
- Admin monitoring shows live vehicle positions

Database / Laravel Structure

- Run all migrations on production DB (dry-run first)
- Seed production data: routes, vehicle master list, initial admin user
- Back up migration state before go-live
- Confirm DB user has minimal required permissions (not root)

Real-time / GPS

- Confirm GPS polling or WebSocket is stable over 30-minute test window
- Verify vehicle_locations updates propagate to commuter map within 6s

Code Quality

- composer install --no-dev for production build
- php artisan config:cache + route:cache + view:cache
- Remove all debug routes (e.g. /api/debug/*, /api/test/*)
- Ensure error reporting goes to log file, not to API response body

Validation Checklist

- Full E2E test: hail at 500m succeeds, at 1500m rejected
- GCash payment flow completes without wallet side effect
- All role-protection tests pass (commuter, conductor, admin isolation)
- No wallet logic in DB, routes, or services

- APP_DEBUG=false (no stack traces in responses)
- Production .env has real PayMongo live keys
- Backup verified before go-live

Final Assessment

Backend Readiness Score (Post-Sprint 9)

Area	Status After Sprint 9	Score
Authentication	Sanctum + role middleware fully enforced	✅ 10/10
GPS System	Real-time upsert with staleness protection	✅ 9/10
Hail Radius (1KM)	100% server-side Haversine enforcement	✅ 10/10
Payment (GCash QR)	PayMongo direct, webhook verified, no wallet	✅ 9/10
Role-Based Access	Middleware on all routes, cross-role tested	✅ 10/10
Wallet Removal	Zero wallet logic in DB, code, or routes	✅ 10/10
API Consistency	Standardized response format, pagination	✅ 9/10
Code Architecture	Thin controllers, service layer, no God classes	✅ 9/10
Security Hardening	Rate limiting, CSRF, OWASP basic audit	✅ 9/10
OVERALL	Production-ready backend	✅ 95/100

Risks if Sprint is Delayed

- Sprint 1 delay (auth/foundation): cascades all subsequent sprints — every sprint depends on it
- Sprint 2 delay (GPS/shifts): Sprint 3 hail logic cannot be tested without live vehicle locations
- Sprint 3 delay (hail): 1KM restriction remains frontend-only — critical security gap
- Sprint 4 delay (payments): GCash QR flow untested — payment failures at launch
- Sprint 7 delay (hardening): wallet ghost logic may ship to production — architectural contamination
- Sprint 8 delay (staging): no pre-production testing environment — production becomes first real test

Deployment Recommendation

Phase	Timeline	Gate
Development	Week 1–7 (June 16 – August 3)	All sprints green
Staging	Week 8 (August 4–10) — separate DB, sandbox PayMongo	E2E tests pass

Production	August 11–15 — after Sprint 9 sign-off	Security audit done

Production go-live is only recommended after Sprint 9 validation checklist is 100% complete. Partial deployments (e.g. going live before hail radius is backend-enforced) introduce critical security and logic gaps.

Critical Architecture Warnings

- NEVER deploy with APP_DEBUG=true — full stack traces expose DB structure and server paths
- The frontend middleware.ts reads the chatco_session cookie role claim client-side — this is display routing only. The backend must ALWAYS re-verify the role via auth:sanctum + role middleware on every request.
- The gcash-payment.ts service has a sandbox fallback that simulates payments (isSimulated: true). Confirm PAYMONGO_CONFIG.isSandbox = false in production .env — the backend must never trust simulated payment results.
- shouldUseConductorApi() in the frontend defaults to API mode unless NEXT_PUBLIC_CONDUCTOR_API_MODE=local is set. Confirm this variable is NOT set in production (it would silently disable all conductor API calls).
- vehicle_locations must be an upsert (INSERT OR UPDATE by vehicle_id), not an append — otherwise the table grows unbounded and GET /api/vehicles/locations returns stale duplicates.
- The frontend's nearby-detector.ts correctly implements all units visible + 1KM hail gate logic, but this is UI-layer only. Sprint 3 backend must duplicate this exact logic in HailService — any divergence is a security gap.